

Yes. If by “**Oax/Alpha/OX**” you mean an AI model/workflow that can inspect a full set of manga/manhwa images and turn them into a **recap script**, the best approach is **not** to ask it “summarize these images” in one shot.

You want a pipeline:

Images → page understanding → character/story continuity → scene-by-scene plan → narration script → QA

For your ManhwaRecap workflow, I would structure the prompt like this.

1. Give the model all pages/images

Upload the pages in **correct reading order**, preferably:

001.jpg
002.jpg
003.jpg
...
021.jpg



If there are 100+ pages, process them in batches, but maintain the page numbers.

2. Use this master prompt

You are an expert Manhwa/Manhua recap scriptwriter and visual story analyst.

I will provide a sequence of manga/manhwa pages in reading order.

Your job is to understand the COMPLETE STORY from the images and create a natural English narration script for a YouTube Manhwa Recap video.

IMPORTANT:

- Analyze EVERY provided page.
- Do NOT summarize only individual pages.
- Connect events across pages and understand the overall story.
- Preserve chronological order.
- Identify recurring characters and keep their names consistent.
- Read speech bubbles, narration boxes, system windows, signs, captions, and important visual information.
- Use the images as the primary source of truth.
- Do not invent events that are not supported by the images.
- If dialogue is unclear, infer only when the surrounding scene makes the meaning obvious.
- If something genuinely cannot be determined, mark it as [UNCLEAR] rather than inventing it.
- Do not describe every panel mechanically. Turn the panels into an engaging story.

STORY ANALYSIS

First determine:

1. Main protagonist
2. Other important characters
3. Setting/world
4. Protagonist's original situation
5. Major events
6. Powers/abilities/system mechanics
7. Main conflict
8. Important twists
9. Emotional reactions
10. Final cliffhanger

Then create a chronological scene structure.

For every scene provide:

- Scene number
- Page range
- What happens
- Characters involved
- Important dialogue/information
- Why the scene matters
- Transition to the next scene

RECAP WRITING STYLE

Write like a professional YouTube Manhwa Recap narrator.

The narration should feel like someone telling an exciting story to the viewer, NOT like someone describing comic panels.

BAD:

"On page 5, Kim walks into the room. There is a man sitting at a desk."

GOOD:

"Kim had no idea that his peaceful vacation was about to come crashing down. The moment he entered the room, his father was already waiting for him."

Use:

- Natural conversational English
- Strong storytelling
- Clear explanations
- Suspense
- Humor when appropriate
- Emotional reactions
- Smooth transitions
- Occasional viewer hooks

Avoid:

- Robotic narration
- Repeating "then"
- Excessive filler
- Describing camera angles
- Saying "in this panel"

- Saying "the image shows"
- Overusing character names
- Inventing dialogue
- Overexplaining obvious visual details

IMPORTANT NARRATIVE RULE

Do not simply repeat the dialogue from the manhwa.

Instead:

1. Explain what the character says.
2. Explain what it means for the story.
3. Connect it to what happened before.
4. Move naturally into the next event.

Use direct dialogue only when it makes the scene stronger.

HOOK

Start the video with a strong 2–4 sentence hook.

The hook should introduce the protagonist's unusual situation and create curiosity about what happens next.

Do NOT reveal the entire story in the opening.

Example style:

"He only wanted one thing after years of working himself to death — an easy life.

And somehow, he got exactly what he wished for.

The problem?

He was reborn as one of the most useless characters in a fantasy world... and accidentally became ridiculously powerful."

Do not copy this example. Create an original hook based on the actual pages.

STORY FLOW

Structure the final narration approximately like this:

HOOK

↓

PROTAGONIST'S ORIGINAL LIFE

↓

IMPORTANT BACKSTORY

↓

TRANSMIGRATION / MAJOR INCIDENT

↓

NEW WORLD

↓

NEW ABILITIES / SYSTEM

↓

FIRST MAJOR PROBLEM

↓

CHARACTER INTERACTIONS

↓

CONFLICT

↓

TWIST

↓

CLIFFHANGER

Do not force this structure if the actual story follows a different sequence.

PACING

Keep the pacing suitable for a YouTube recap.

Fast-moving scenes:

Use short, punchy narration.

Emotional scenes:

Slow down and explain the character's situation.

Action scenes:

Focus on what happens and why it matters rather than describing every attack.

System/power scenes:

Clearly explain what the ability does and why it is important.

OUTPUT FORMAT

Return the result in these sections:

1. STORY OVERVIEW

Give a concise overview of the entire chapter.

2. CHARACTER LIST

Character	Role	Important traits
-----------	------	------------------

3. CHRONOLOGICAL STORY PLAN

Scene	Pages	Events	Important information
-------	-------	--------	-----------------------

4. FINAL YOUTUBE RECAP SCRIPT

Write the complete narration as a continuous story.

Do not include page numbers inside the narration.

Use paragraph breaks between major scenes.

5. SCENE-TO-SCRIPT MAPPING

After the script, provide:

Script section	Source pages	Key visual/event
----------------	--------------	------------------

This is extremely important because the script will later be synchronized with the manhwa images.

6. FACT CHECK / UNCERTAINTIES

List anything that:

- was difficult to read
- had ambiguous character identity
- contained unclear dialogue
- required contextual interpretation

Do not invent missing information.

FINAL QUALITY CHECK

Before giving the answer, verify:

- ☐ Every page was considered.
- ☐ Events are in chronological order.
- ☐ Character names are consistent.
- ☐ No major event was skipped.
- ☐ No unsupported event was invented.

- [] System abilities are explained correctly.
- [] The protagonist's motivations are clear.
- [] The story flows naturally.
- [] The script sounds like human narration.
- [] The ending preserves the chapter's cliffhanger.
- [] The script can be synchronized back to the original pages.

Now analyze all uploaded pages and create the complete Manhwa Recap story.

3. But for your workflow, I'd make it even better

The **biggest improvement** is to separate **analysis from writing**.

Don't make the AI immediately write a 20-minute script after looking at 100 images.

Use **3 passes**.

Pass 1 — Vision analysis

Ask:

"Understand every page and create a structured story database. Do not write the final narration yet."

Output:

Page 1:
Characters:
Events:
Dialogue:
Important visual information:
Story purpose:

Page 2:
...



This prevents the model from forgetting pages.

Pass 2 — Story planning

Then give it the extracted analysis and say:

Now connect all pages into one continuous story.



Identify:

- protagonist
- supporting characters
- world
- timeline
- powers
- conflicts
- twists
- motivations
- cliffhangers

Remove repetitive information.

Create a scene-by-scene recap plan.

Every scene MUST contain its source page numbers.

You might get something like:

Scene 1 – Pages 1–4

Kim's new identity is introduced.



Scene 2 – Pages 5–8

Flashback explains his previous life as a salesman.

Scene 3 – Pages 9–12

He accepts the hidden scenario and transmigrates.

Scene 4 – Pages 13–17

His hidden abilities activate.

Scene 5 – Pages 18–21

His father announces his expulsion.

This is extremely useful for your video editing.

4. Pass 3 — Generate the actual voice-over

Then use the story plan:

Using the approved story plan, write the final YouTube Manhwa Recap narration.



Target:

- Natural American/global English
- Human storyteller tone
- Fast but understandable pacing
- No robotic wording
- No panel descriptions

- No unnecessary dialogue
- Explain important events clearly
- Maintain suspense
- Use humor naturally when appropriate

The narrator should sound like someone telling a friend an incredible story.

Do not say:

"On this page..."

"In this panel..."

"The image shows..."

Instead explain what is happening naturally.

For every scene, provide:

Scene:

Source pages:

Narration:

Approximate duration:

Target word count:

At the end provide:

Total word count

Estimated narration duration

5. For your specific example

Your current story could become something like this:

HOOK

Kim spent years becoming the ultimate sales machine. He worked himself to the bone, climbed to the top, and made enough money to retire early. All he wanted after that was a peaceful life where nobody could bother him.

And somehow, that ridiculous dream actually came true.

Unfortunately for Kim, it came with one tiny problem.

He had been reincarnated into the body of the most useless noble in a fantasy world.

Then transition:

Before becoming Van Ecclesia, Kim was already a legend in the corporate world...

Then explain his sales career.

Then:

But while playing his favorite RPG, Kim couldn't understand why everyone loved the heroic protagonists. They were constantly fighting monsters, risking their lives, and suffering through tragedy just to become stronger.

Kim had a much better candidate for himself.

Van Ecclesia.

He was practically useless when it came to magic, but as the legitimate heir of a powerful ducal family, Van had everything Kim had ever wanted — money, servants, luxury, and absolutely no reason to work.

Then the hidden scenario:

And just when Kim thought the game was over, a strange notification appeared.

SALVATION OF THE WORLD.

It was a hidden scenario he'd never seen before.

Naturally, Kim accepted it.

He probably should have thought that decision through.

Then transmigration.

Then the system/powers.

Then comedy:

At first, Van thought his new life was perfect.

For an entire week, he did absolutely nothing productive.

He ate expensive food, enjoyed luxurious baths, gambled, and basically treated the entire duchy like his personal vacation resort.

Finally...

Kim had achieved his dream.

Or so he thought.

Then the power reveal:

Everything changed when he opened his status window.

His supposedly useless **Mana Insensitivity** suddenly revealed a hidden potential.

Van casually selected **YES**.

And that's when things got completely ridiculous.

Instead of receiving some minor ability, he unlocked **Grandmaster of Magic**.

And apparently, that wasn't even the end.

Then the comedy with the maid:

Unfortunately, Van soon discovered that being overwhelmingly powerful came with a few side effects.

His new charisma wasn't exactly... friendly.

When a frightened maid fell in front of him, Van simply wanted to ask if she was okay.

But the terrifying aura surrounding him made it sound like he was about to execute her.

And finally:

But before Van could enjoy another peaceful day, his father summoned him.

The Duke of Ecclesia wasn't someone Van wanted to disappoint.

So he nervously entered the enormous office, hoping this was nothing serious.

Then his father looked directly at him and dropped a sentence that completely destroyed Van's peaceful fantasy.

"I've decided to expel you from the ducal family."

Van's entire world froze.

Because if he was no longer a member of the Ecclesia family...

his dream life was over.

That is the kind of narration you want the model to produce: the images provide the facts, while the AI turns those facts into a continuous story.

6. The important part for your YouTube workflow

Since you're already working with **downloaded/manhwa source images** and **recap videos**, I'd actually create a structured intermediate file like:

```
project/
|
├─ images/
|   ├── 001.jpg
|   ├── 002.jpg
|   └─ ...
|
├─ 01_page_analysis.json
├─ 02_story_plan.json
├─ 03_recap_script.txt
├─ 04_voiceover.json
└─ 05_editing_map.json
```

And the final editing map could look like:

```
{
  "scene": 5,
  "pages": [18, 19, 20, 21],
  "narration": "But before Van could enjoy another peaceful day...",
  "duration_sec": 24,
  "visual_order": [
    "018.jpg",
    "019.jpg",
    "020.jpg",
    "021.jpg"
  ]
}
```

That gives you a direct pipeline:

Manhwa images → AI understands story → story plan → human-like script → voice-over → image synchronization → final recap video

For your use case, **the key isn't just getting a good summary**. The key is preserving the **page-to-script mapping**, because that lets you automatically create the final recap video later.

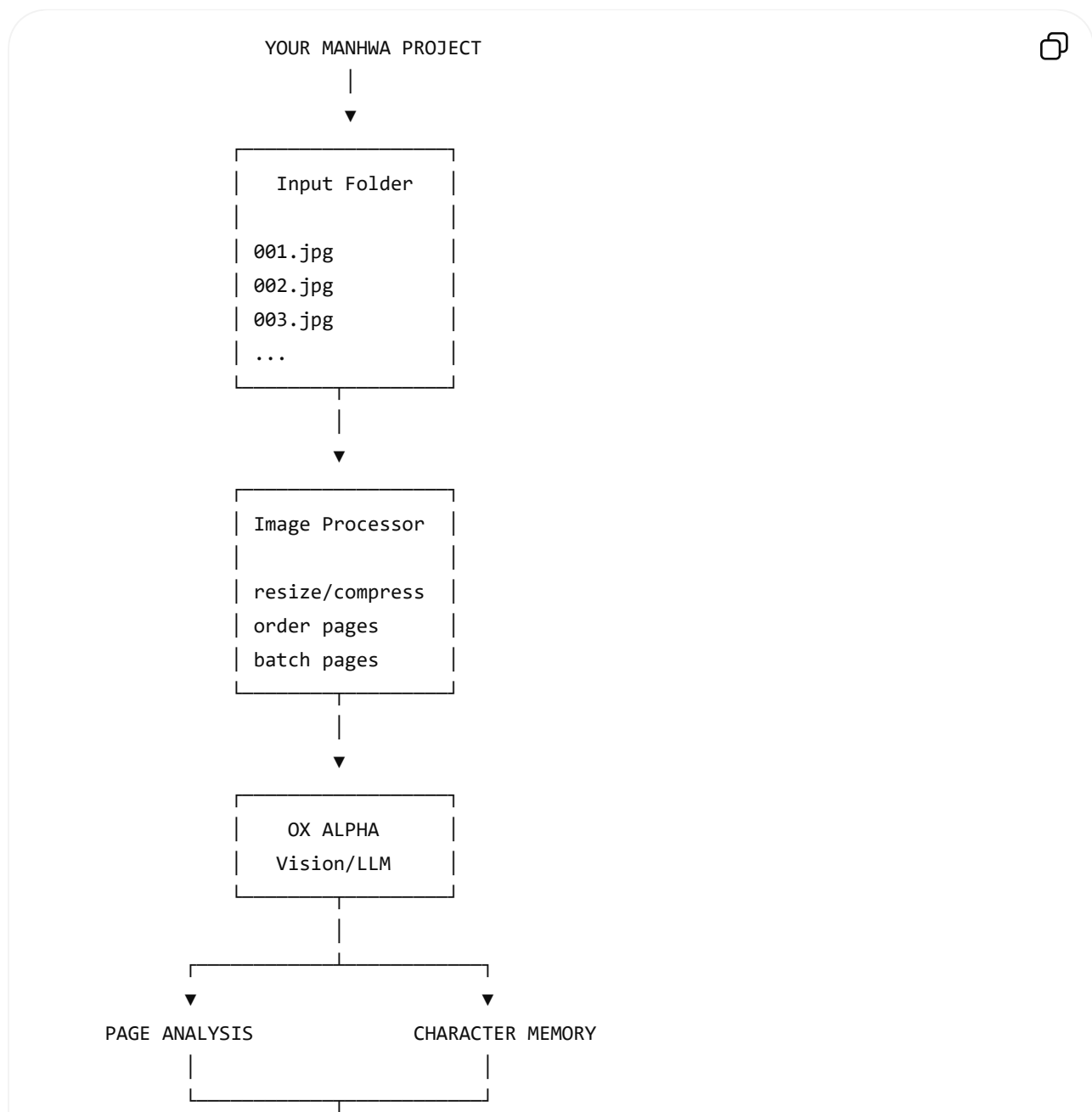


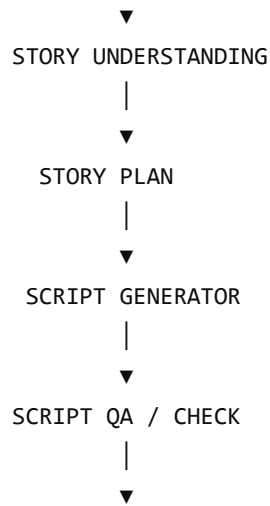
above is good but we need alpha ox api to connect so we can do automation

Yes — **if you mean Ox Alpha (ox-alpha)**, then this is exactly the kind of automation we can build. I found its current API documentation: it provides an **OpenAI-compatible API**, with base URL `https://oxalpha.run/api/v1`, model ID `ox-alpha`, and Bearer-key authentication.  Ox Alpha

The important thing is that we should build **your own image → story → script pipeline** around the Ox Alpha API rather than manually doing the analysis each time.

The automation I recommend





```
final_script.json
final_script.txt
scene_map.json
```

Why this is better

Instead of:

Upload images → "write story"

we make Ox Alpha perform **multiple controlled jobs**.

That will be much more reliable for your 50–200 page Manhwa recap projects.

1. First: connect Ox Alpha

Their API is OpenAI-compatible, so the architecture can use the standard OpenAI SDK pattern while changing the endpoint. Their documentation currently specifies:  Ox Alpha

Base URL:
`https://oxalpha.run/api/v1`

Model:
`ox-alpha`

Authentication:
`Bearer YOUR_API_KEY`



For example, in Python:

`</>` Python



`▶` Run

```

from openai import OpenAI
import os

client = OpenAI(
    api_key=os.environ["OX_ALPHA_API_KEY"],
    base_url="https://oxalpha.run/api/v1"
)

response = client.chat.completions.create(
    model="ox-alpha",
    messages=[
        {
            "role": "user",
            "content": "Analyze this story..."
        }
    ]
)

print(response.choices[0].message.content)

```

The official API page also shows the equivalent cURL and Node/OpenAI-SDK approach.



2. But there is one important issue

Your task isn't just text generation.

You need:

images → visual understanding

So before implementing the automation, we need to verify exactly how the current Ox Alpha API accepts **image inputs**.

The API documentation I found confirms the OpenAI-compatible chat endpoint, but the page I found doesn't establish the exact current image-input schema.  Ox Alpha

So I would make the application have an abstraction like:

 **Python**



 **Run**

```

class VisionProvider:

    def analyze_images(self, images, prompt):
        pass

```

Then:

</> Python



▶ Run

```
class OxAlphaProvider(VisionProvider):  
  
    def analyze_images(self, images, prompt):  
        # Ox Alpha implementation  
        pass
```

This is important because if Ox Alpha's current endpoint handles images differently, we only change this adapter rather than rebuilding the whole application.

3. Your actual application

I'd build this as a Python desktop automation tool first.

Something like:

Manhwa Recap Generator



Project

[The Creed of the Sales King]

Images

[127 images detected]

[Analyze Images]

Progress

 78%

Pages analyzed: 99 / 127

↓

Story Understanding

- ✓ Characters
- ✓ Timeline
- ✓ Powers
- ✓ Events
- ✓ Conflicts

↓

[Generate Story Plan]

↓

[Generate Script]



Final Output

- ✓ story_analysis.json
- ✓ story_plan.json
- ✓ recap_script.txt
- ✓ recap_script.json
- ✓ scene_mapping.json

Eventually you could turn this into your own web application.

4. Image analysis should work in batches

Suppose you have:

001.jpg
002.jpg
003.jpg
...
150.jpg



Don't send all 150 pages blindly in one request.

Use:

Batch 1 → 001-015
Batch 2 → 016-030
Batch 3 → 031-045
...



Each batch produces structured data.

For example:

</> JSON



```
{  
  "pages": [1, 2, 3, 4, 5],  
  "characters": [  
    {  
      "name": "Kim",  
      "description": "Modern salesman"  
    }  
  ]  
}
```

```

],
"events": [
  {
    "page": 1,
    "event": "Citizens discuss a mysterious hero"
  },
  {
    "page": 2,
    "event": "Kim's identity is revealed"
  }
],
"dialogue": [],
"important_details": []
}

```

5. Then maintain a CHARACTER MEMORY

This is **very important**.

Otherwise the AI might call the same character:

```

Kim
Van
Van Ecclesia
the blonde man
the protagonist
the Duke's son

```



as six different characters.

We maintain:

```

</> JSON

{
  "characters": {
    "kim": {
      "aliases": [
        "Kim",
        "Van",
        "Van Ecclesia"
      ],
      "role": "main protagonist",
      "original_identity": "modern salesman",
      "current_identity": "Van Ecclesia"
    },

    "duke_ecclesia": {

```



```

    "aliases": [
      "Duke Ecclesia",
      "Hjam Ecclesia",
      "father"
    ],
    "role": "protagonist's father"
  }
}

```

Every later AI request receives the relevant character memory.

6. Then the STORY PLANNER

After all image batches have been analyzed:

```

IMAGE ANALYSIS
  ↓
COMBINE
  ↓
REMOVE DUPLICATES
  ↓
RESOLVE CHARACTERS
  ↓
BUILD TIMELINE
  ↓
STORY PLAN

```



Example:

```

</> JSON

{
  "scenes": [
    {
      "scene_id": 1,
      "pages": [1, 2, 3],
      "title": "The mysterious hero",
      "events": [
        "Citizens discuss the Ascendant",
        "The hero's achievement is revealed",
        "Kim is secretly watching"
      ]
    },
    {
      "scene_id": 2,
      "pages": [4, 5, 6, 7],
      "title": "Kim's old life",

```



```
    "events": [
      "Kim is introduced as a salesman",
      "He wants early retirement",
      "He plays Hero Academy"
    ]
  }
]
```

7. Then generate the actual recap

Now Ox Alpha receives:

```
STORY PLAN
+
CHARACTER MEMORY
+
YOUR WRITING STYLE
+
YOUR RECAP RULES
```



and produces:

```
</> JSON

{
  "scene_id": 1,
  "pages": [1, 2, 3],
  "narration": "Kim had spent years working...",
  "estimated_words": 96,
  "estimated_duration": 38
}
```



This is much better than generating one giant TXT file.

8. And this is where it becomes REALLY useful for your videos

Because we preserve:

```
SCRIPT
↓
SCENE
```





PAGE RANGE

you can later automatically generate:

Scene 1

Narration: 00:00-00:38

Images: 001-003

Scene 2

Narration: 00:38-01:21

Images: 004-009

Scene 3

Narration: 01:21-02:05

Images: 010-015



Then your video-generation system can automatically place the correct manhwa images underneath the voice-over.

9. I would also add a "Human Tone" prompt

Because this is important for your channel.

Your system prompt should contain something like:

The narration must NOT sound like an AI-generated summary.

Write like a human YouTube storyteller who is excitedly explaining an interesting manh

Use natural sentence variation.

Avoid repetitive sentence structures.

Do not constantly use:

"Meanwhile..."

"However..."

"Then..."

"After that..."

Use character reactions and context naturally.

Do not narrate every visual detail.

Prioritize:

1. What happened
2. Why it matters
3. What the character thinks/wants



4. What changed
5. What the viewer should be curious about next

The narrator should sound conversational, confident and entertaining.

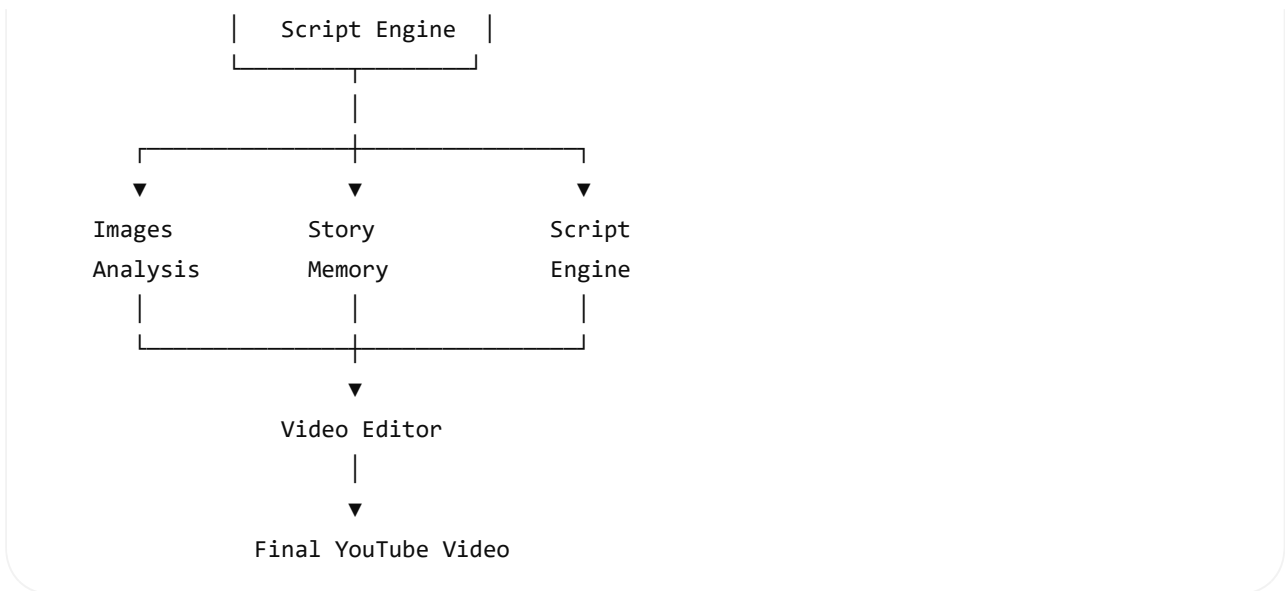
10. Final architecture

For your project, I'd build:

```
Python
|
├─ input/
|   └─ manhwa_project/
|       ├── 001.jpg
|       ├── 002.jpg
|       └─ ...
|
├─ analyzer/
|   ├── image_processor.py
|   ├── batch_manager.py
|   └─ ox_alpha.py
|
├─ story/
|   ├── character_manager.py
|   ├── timeline.py
|   ├── story_planner.py
|   └─ continuity_checker.py
|
├─ script/
|   ├── script_generator.py
|   ├── tone_manager.py
|   └─ script_qa.py
|
├─ output/
|   ├── page_analysis.json
|   ├── characters.json
|   ├── story_plan.json
|   ├── recap_script.txt
|   ├── recap_script.json
|   └─ scene_mapping.json
|
└─ main.py
```

And later:

```
┌───────────┐
│ ManhwaRecapGod │
```



One correction from my previous answer

I initially interpreted your wording as potentially referring to **alphaXiv**, but that is a different service for research/arXiv papers. Its API/MCP is for research-paper retrieval, not this image-to-Manhwa workflow.  alphaXiv +1

Ox Alpha is the service you appear to mean, and its current API specifically documents the `ox-alpha` model and OpenAI-compatible endpoint.  Ox Alpha

If you're building this on your Windows machine, I can next give you the [↳ complete Python project that takes a folder of 100+ JPG/PNG pages → calls Ox Alpha → analyzes the images → builds character memory → generates the story plan → produces the final recap script JSON/TXT automatically.](#)



how to get api key

Yes. There are **two different ways** to get an Ox Alpha API key right now, depending on which API route you want to use.

Option 1 — Direct Ox Alpha API

The official Ox Alpha API page says:

1. Go to [Ox Alpha API](#)
2. Sign in.
3. Open the **Developer Console**.
4. Create an API key.

5. Copy the secret key — it is shown only once.  Ox Alpha

The direct API uses:

```
Base URL:
https://oxalpha.run/api/v1

Model:
ox-alpha

Authentication:
Bearer YOUR_API_KEY
```



It currently documents a **3 million token/day free tier** and up to **12 images per request**, which is particularly interesting for your Manhwa automation.  Ox Alpha

Option 2 — OpenRouter

You can also access Ox Alpha through OpenRouter using:

```
Model:
stealth/ox-alpha

Base URL:
https://openrouter.ai/api/v1
```



You would:

1. Create an account on [OpenRouter](#) ↗
2. Open the API Keys section.
3. Click **Create Key**.
4. Copy the `sk-or-v1-...` key.
5. Use `stealth/ox-alpha` as the model.  OXAlpha +1

For your Manhwa project

I'd start with **the direct Ox Alpha API**, because your workflow is specifically:

12 images/request → analyze → maintain story memory → next 12 images → generate final script.

The documented direct API already supports image inputs and gives you the 3M-token daily free tier.  Ox Alpha

Don't paste your API key here. Put it in your local `.env` file:

```
</> env
```



```
OX_ALPHA_API_KEY=your_key_here
```

Then we can build the automation around it.

If you want, ↳ the next step can be the **actual Windows Python project**: ↳ you put 100–200 manhwa images into a folder, run `python main.py`, and it automatically sends batches to Ox Alpha and creates `story_plan.json`, `recap_script.txt`, and `scene_mapping.json`.



i have ox api key from develop console what next step

Perfect. Since you already have the **Ox Alpha API key**, the next step is **not to build the whole automation immediately**. First, let's verify that your key works from your Windows PC. The current Ox Alpha API is OpenAI-compatible, uses `https://oxalpha.run/api/v1`, and the model is `ox-alpha`.  Ox Alpha

Step 1 — Create the project

On Windows:

```
</> PowerShell
```



```
mkdir manhwa-recap-ai
cd manhwa-recap-ai

python -m venv venv
venv\Scripts\activate

pip install openai python-dotenv
```

Your folder should become:

```
manhwa-recap-ai/
|
├─ venv/
├─ .env
└─ test_ox.py
```



Step 2 — Put your API key in .env

Create:

.env



Put:

</> env



```
OX_ALPHA_API_KEY=YOUR_ACTUAL_API_KEY_HERE
```

Don't send the key to me or put it directly into your Python code.

Step 3 — Test Ox Alpha

Create:

test_ox.py



with:

</> Python



▶ Run

```
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

api_key = os.getenv("OX_ALPHA_API_KEY")

if not api_key:
    raise ValueError("OX_ALPHA_API_KEY not found in .env")

client = OpenAI(
    api_key=api_key,
    base_url="https://oxalpha.run/api/v1"
)

response = client.chat.completions.create(
    model="ox-alpha",
    messages=[
        {
            "role": "user",
            "content": "Explain in one sentence what a manhwa recap is."
        }
    ]
)
```

```
]
)

print(response.choices[0].message.content)
```

This follows Ox Alpha's current OpenAI-compatible API structure.  Ox Alpha

Step 4 — Run it

```
</> PowerShell

python test_ox.py
```



You should get something similar to:

A manhwa recap summarizes and narrates the important events of a Korean comic in an en



If you get that, **your API connection is working.**

Step 5 — Then we'll test IMAGE understanding

This is the important part for your actual project.

Your API currently documents support for **up to 12 images per request.**  Ox Alpha

So we'll create:

```
manhwa-recap-ai/
|
├─ images/
|   ├── 001.jpg
|   ├── 002.jpg
|   ├── 003.jpg
|   ├── ...
|   └─ 012.jpg
|
├─ output/
|
├─ .env
├─ test_ox.py
└─ analyze_images.py
```



Then `analyze_images.py` will do:

```
001.jpg ┌
002.jpg │
003.jpg │
004.jpg │
...      └───> 0x Alpha
012.jpg ┌

      ↓
    Page Analysis
      ↓
    JSON file
```

For example:

</> JSON

```
{
  "pages": [
    {
      "page": 1,
      "events": [
        "Citizens discuss a mysterious hero"
      ],
      "characters": [
        "unknown blonde man"
      ]
    },
    {
      "page": 2,
      "events": [
        "The protagonist is revealed"
      ]
    }
  ]
}
```

Step 6 — Then automate the entire thing

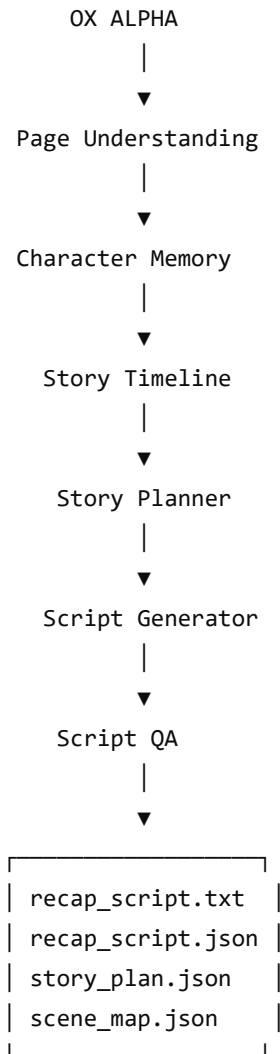
Once the image test works, we'll build this:

MANHWA IMAGES



Image Batch
12 at a time





And because Ox Alpha's API supports JSON output via `response_format` , we can make the intermediate results structured rather than relying on messy plain text. 🐻 Ox Alpha

One thing to keep in mind

The API currently documents **3,000,000 free tokens/day**, **60 requests/minute**, and **12 images/request**. 🐻 Ox Alpha That makes batching important for your 100–200 page chapters.

So do Step 1–4 first. Once `python test_ox.py` successfully returns an answer, the next piece is the **actual `analyze_images.py` that sends your 12 Manhwa pages to Ox Alpha and produces the first structured story analysis.**



Files, images, and data analysis are unavailable until usage resets at 10:21 PM. Continue chatting with text only, or upgrade for more access.

✦ Upgrade to Plus

ChatGPT can make mistakes. Check important info.